Session Management Service with Aerospike

This project delivers a high-performance session management service using FastAPI and Aerospike, a distributed NoSQL database built for real-time workloads.

Project Overview

1

One-Session-Per-User

Enforces a single active session per user, preventing sprawl and enabling "log out everywhere."

2

Cookie-Based Sessions

Utilizes starssessions middleware for secure, HMAC-signed, and configurable cookie handling.

3

Dual Interface

Offers both an HTML UI for browsers and a JSON REST API for programmatic session management.

4

Dockerized Deployment

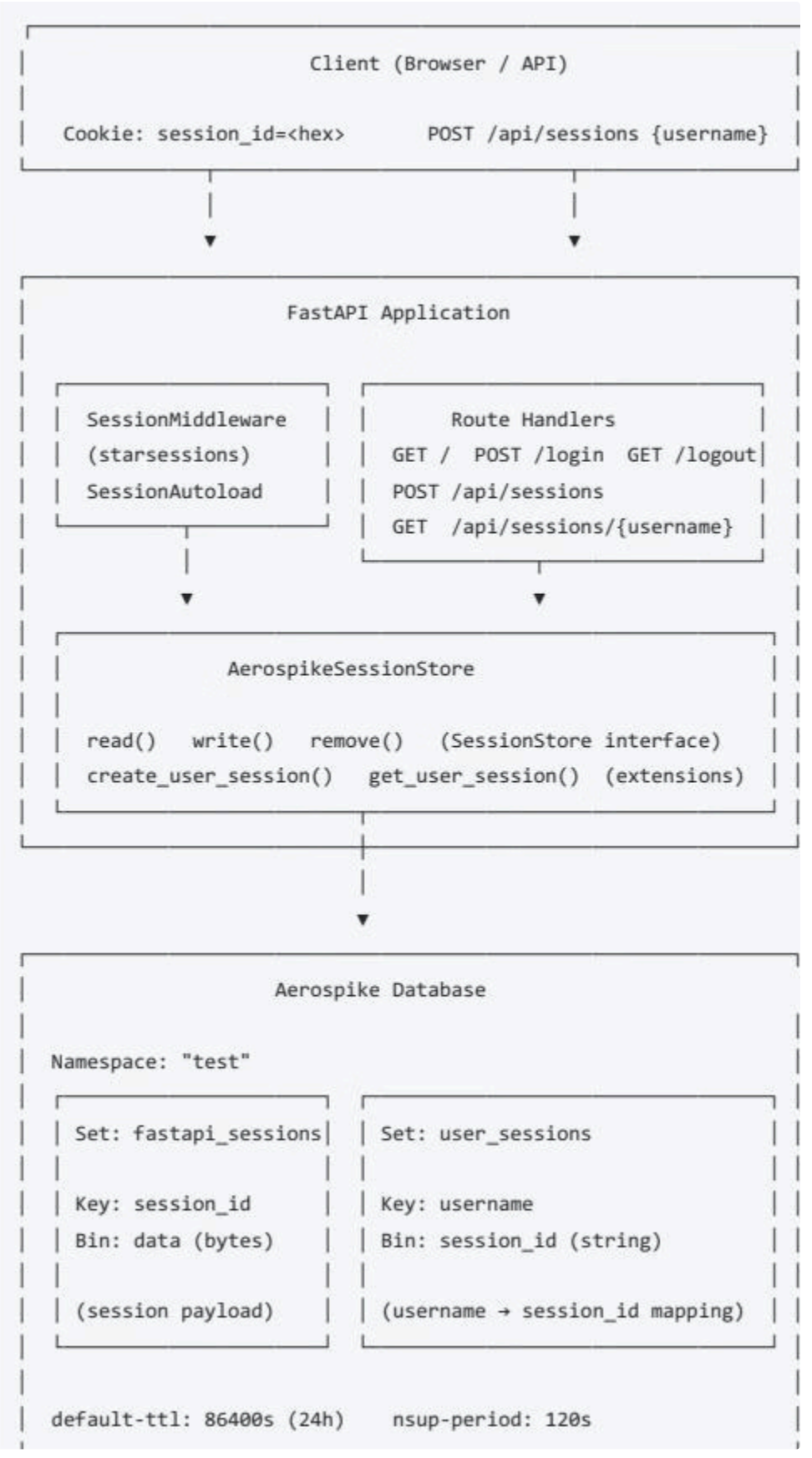
Simple setup with Docker Compose for both Aerospike and the FastAPI application.

5

TTL-Driven Expiry

Aerospike's native Time-To-Live (TTL) automatically expires session records, eliminating manual garbage collection.

Architecture & Core Components



Request Flow

01

Client Request

Client sends request with or without a session_id cookie.

02

Middleware Interception

SessionMiddleware extracts session ID and loads data via AerospikeSessionStore.read().

03

Route Handler Access

Route handler uses request.session to read/write session data.

04

Data Persistence

Middleware calls AerospikeSessionStore.write() to save changes, enforcing one-session-per-user.

AerospikeSessionStore: The Core

This component extends `starssessions.SessionStore`, implementing essential methods like `read()`, `write()`, and `remove()`, along with custom extensions for user session management.

API Reference & Data Model

JSON API Endpoints

POST /api/sessions

Creates a new session, invalidating any previous one for the user.

```
{ "username": "alice" }
```

GET /api/sessions/{username}

Retrieves the active session for a user.

```
{ "session_id": "...", "data": { "username": "alice" }, "ttl": 85231 }
```

Aerospike Data Model

Set: fastapi_sessions	Description
Key	Session ID (32-char hex string)
Bin: data	JSON-serialized session payload
TTL	Managed by Aerospike namespace default-ttl

Set: user_sessions	Description
Key	Username
Bin: session_id	Active session ID for this user
TTL	Managed by Aerospike namespace default-ttl

This bidirectional mapping ensures efficient session-to-user and user-to-session lookups.

Deployment & Configuration

Quick Start

Prerequisites: Docker and Docker Compose.

```
cd /path/to/SessionStore
docker compose up --build
```

This command starts Aerospike CE 8.1.1.1 (ports 3000–3002) and the FastAPI app (port 8001).

Visit http://localhost:8001/login to log in.

Environment Variables

Variable	Default	Description
AEROSPIKE_HOST	127.0.0.1	Aerospike server hostname
AEROSPIKE_PORT	3000	Aerospike server port

Aerospike Namespace Configuration

Parameter	Value	Description
NAMESPACE	test	Namespace for session data
DEFAULT_TTL	86400	Record TTL in seconds (24 hours)
NSUP_PERIOD	120	Namespace supervisor cycle interval

FastAPI Session Configuration

Parameter	Value	Description
SESSION_LIFETIME	86400	Session lifetime (seconds)
cookie_https_only	False	Set to True in production with TLS
cookie_name	session_id	Name of the session cookie

Production Readiness Roadmap

Transforming this project into a production-grade session service requires addressing key areas:

Authentication

Replace the hardcoded user database with a robust identity provider (e.g., OAuth 2.0, LDAP, database-backed auth).

Session Security

Enable HTTPS-only cookies, set samesite attributes, sign session IDs, and rotate IDs on privilege escalation.

API Authentication

Secure API endpoints with API keys, JWTs, or mutual TLS for public-facing deployments.

Async Aerospike Client

Implement true non-blocking I/O for high concurrency using `asyncio.to_thread()` or an async client.

Connection Pooling

Tune `max_conns_per_node` for multi-worker deployments to handle high-concurrency workloads.

Observability

Add structured logging, Prometheus metrics, distributed tracing (OpenTelemetry), and health checks.

Rate Limiting

Protect login and session creation endpoints from brute-force attacks with rate limiters and progressive delays.

Graceful Shutdown

Ensure proper handling of SIGTERM in containerized environments to drain requests before closing connections.

Scaling Strategies: Maintaining p90 < 10ms

Aerospike's design ensures predictable, low-latency access for session management at internet scale. The goal is to maintain p90 response times in single-digit milliseconds across all tiers.

Latency Budget Breakdown

Component	Budget	Total
Network hop (client → app)	1–3ms	~3–7ms
App processing	0.5–1ms	
Aerospike read/write	0.5–4ms	
Network hop (app → Aerospike)	0.2–1ms	

Scaling Tiers

1

Tier 1: 1,000 Users

Single Aerospike node, 1-2 FastAPI workers. Default config is sufficient. Expected p90: < 2ms.

2

Tier 2: 100,000 Users

1-2 Aerospike nodes (RF2), 4-8 FastAPI workers. In-memory storage with persistence, multiple Uvicorn workers, `asyncio.to_thread()`. Expected p90: < 3ms.

3

Tier 3: 1,000,000 Users

3-4 Aerospike nodes (RF2), 8-16 FastAPI instances (Kubernetes HPA). Aerospike cluster sizing, client policy tuning. Expected p90: < 5ms.

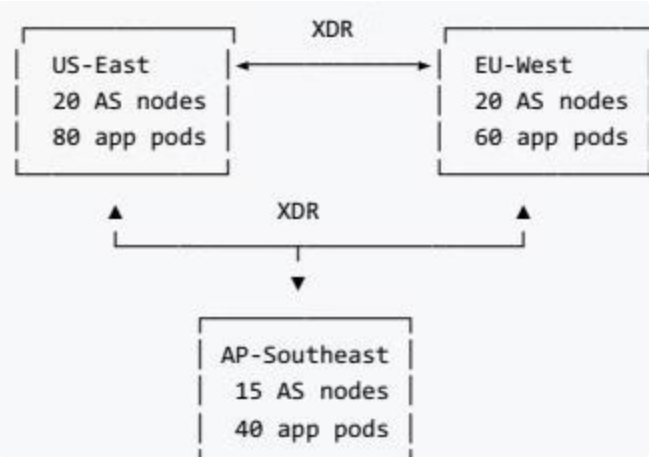
Scaling Strategies: Tiers 4-6

Tier 4: 10,000,000 Users

5-8 Aerospike nodes (NVMe SSDs), 32-64 FastAPI instances. Hybrid storage (index in RAM, data on SSD), application-level caching, XDR for disaster recovery. Expected p90: < 5-8ms.

Tier 5: 100,000,000 Users

20-40 Aerospike nodes per region, 100-200+ FastAPI instances. Multi-region active-active with XDR, partition-aware client routing, dedicated worker pools. Expected p90: < 5ms/region.



Tier 6: 1,000,000,000 Users

100-200+ Aerospike nodes per region, 500-1000+ FastAPI instances. Aerospike All-Flash with Index on flash, shard by user geography, hot-key mitigation, tiered storage. Expected p90: < 8ms/region.

```
User login → GeoIP lookup → Route to regional cluster
                                    |
                                    ▼
                          Regional Aerospike Cluster
                          (owns this user's session)
```

Scaling Summary & Universal Principles

Scale	Aerospike Nodes	App Instances	Storage	Data Size	Expected p90
1K	1	1-2	Memory	~1MB	< 2ms
100K	1-2	4-8	Memory	~100MB	< 3ms
1M	3-4	8-16	Memory / SSD	~1GB	< 5ms
10M	5-8	32-64	NVMe SSD	~10GB	< 5-8ms
100M	20-40/region	100-200	NVMe SSD	~100GB	< 5ms/region
1B	100-200/region	500-1000	NVMe + PMem	~1TB	< 8ms/region

Universal Principles for All Tiers

→ **Small Session Payloads**

Store only identifiers; fetch full profiles separately for faster I/O.

→ **Measure Before Optimizing**

Identify bottlenecks with metrics before adding complexity.

→ **Prefer Reads from Replicas**

Spread read load by configuring clients to prefer nearest replicas.

→ **Avoid Cross-Datacenter Reads**

Route users to the nearest regional cluster; use XDR for async replication.

→ **Fail Open for Session Reads**

Treat temporary unavailability as "no session" and redirect to login.

→ **Automate Capacity Planning**

Monitor usage, set alerts, and scale out proactively as Aerospike rebalances automatically.

Questions?

Thank You